```
#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
#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
!pip install wandb timm --quiet
import wandb
wandb.login()
```

```
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter: ··· ··· ··· ···
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: sundaramsahu111 (sundaramsahu111-dtu) to https://api.wandb.ai. Use `wandb login --relogin` to
True
```

```
from huggingface_hub import notebook_login
notebook_login()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
import torchvision
import torchvision.transforms as transforms
from torchvision.models import resnet18

import numpy as np
import matplotlib.pyplot as plt
import time
import copy
import wandb
from itertools import product as iter_product

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
```

```
Using device: cuda
```

```
# --- Base transforms (no augmentation) ---
transform_base = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# --- Augmented transforms (horizontal + vertical flip) ---
transform_augmented = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# --- Test/Val transform (no augmentation ever) ---
transform_test = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# Download full CIFAR-10
full_train = torchvision.datasets.CIFAR10(root='./data', train=True,
                                          download=True, transform=None)
test_dataset_final = torchvision.datasets.CIFAR10(root='./data', train=False,
                                                  download=True, transform=transform_test)
```

```
# 80/10/10 split from the 50 000 training images
num_total = len(full_train)  # 50000
num_train = int(0.8 * num_total)  # 40000
num_val   = int(0.1 * num_total)  # 5000
num_test  = num_total - num_train - num_val  # 5000

indices = np.random.permutation(num_total)
train_idx = indices[:num_train]
val_idx   = indices[num_train:num_train + num_val]
test_idx  = indices[num_train + num_val:]


class TransformSubset(torch.utils.data.Dataset):
    """Wraps a Subset with a custom transform."""
    def __init__(self, dataset, indices, transform):
        self.dataset = dataset
        self.indices = indices
        self.transform = transform

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, idx):
        img, label = self.dataset[self.indices[idx]]
        if self.transform:
            img = self.transform(img)
        return img, label


def get_dataloaders(augment=False, batch_size=128):
    """Return train, val, test DataLoaders."""
    train_tf = transform_augmented if augment else transform_base
    train_ds = TransformSubset(full_train, train_idx, train_tf)
    val_ds   = TransformSubset(full_train, val_idx,   transform_test)
    test_ds  = TransformSubset(full_train, test_idx,  transform_test)

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                              num_workers=2, pin_memory=True)
    val_loader   = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                              num_workers=2, pin_memory=True)
    test_loader  = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
                              num_workers=2, pin_memory=True)
    return train_loader, val_loader, test_loader


print(f"Train: {num_train} | Val: {num_val} | Test: {num_test}")
```

```
100%|██████████| 170M/170M [00:13<00:00, 12.5MB/s]
Train: 40000 | Val: 5000 | Test: 5000
```

```
class PatchEmbedding(nn.Module):
    """Split image into patches and project to embedding dimension."""
    def __init__(self, img_size=32, patch_size=4, in_channels=3, embed_dim=128):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(in_channels, embed_dim,
                              kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # x: (B, C, H, W) -> (B, num_patches, embed_dim)
        x = self.proj(x)                      # (B, embed_dim, H', W')
        x = x.flatten(2).transpose(1, 2)      # (B, num_patches, embed_dim)
        return x


class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_dim, num_heads, dropout=0.1):
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.scale = self.head_dim ** -0.5

        self.qkv = nn.Linear(embed_dim, embed_dim * 3)
        self.proj = nn.Linear(embed_dim, embed_dim)
        self.attn_drop = nn.Dropout(dropout)
        self.proj_drop = nn.Dropout(dropout)

    def forward(self, x):
        B, N, C = x.shape
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4)     # (3, B, heads, N, head_dim)
        q, k, v = qkv[0], qkv[1], qkv[2]

        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        x = (attn @ v).transpose(1, 2).reshape(B, N, C)
        x = self.proj(x)
        x = self.proj_drop(x)
        return x


class TransformerBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, mlp_ratio=4.0, dropout=0.1):
```

```
        super().__init__()
        self.norm1 = nn.LayerNorm(embed_dim)
        self.attn = MultiHeadSelfAttention(embed_dim, num_heads, dropout)
        self.norm2 = nn.LayerNorm(embed_dim)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, int(embed_dim * mlp_ratio)),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(int(embed_dim * mlp_ratio), embed_dim),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        x = x + self.attn(self.norm1(x))   # Residual + attention
        x = x + self.mlp(self.norm2(x))    # Residual + FFN
        return x


class VisionTransformer(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_channels=3,
                 num_classes=10, embed_dim=128, depth=6,
                 num_heads=4, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.patch_embed = PatchEmbedding(img_size, patch_size,
                                          in_channels, embed_dim)
        num_patches = self.patch_embed.num_patches

        # Learnable CLS token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        # Learnable positional encoding
        self.pos_embed = nn.Parameter(
            torch.zeros(1, num_patches + 1, embed_dim))
        self.pos_drop = nn.Dropout(dropout)

        # Transformer encoder blocks
        self.blocks = nn.Sequential(
            *[TransformerBlock(embed_dim, num_heads, mlp_ratio, dropout)
              for _ in range(depth)])

        self.norm = nn.LayerNorm(embed_dim)

        # Classification head
        self.head = nn.Sequential(
            nn.Linear(embed_dim, embed_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(embed_dim, num_classes),
        )

        # Initialize
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)                              # (B, N, D)
        cls_tokens = self.cls_token.expand(B, -1, -1)        # (B, 1, D)
        x = torch.cat([cls_tokens, x], dim=1)                # (B, N+1, D)
        x = x + self.pos_embed                               # Add position
        x = self.pos_drop(x)
        x = self.blocks(x)                                   # Encoder
        x = self.norm(x)
        cls_out = x[:, 0]                                    # CLS token
        return self.head(cls_out)


def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


# Quick test
vit_test = VisionTransformer().to(device)
print(f"ViT parameters: {count_parameters(vit_test):,}")
dummy = torch.randn(2, 3, 32, 32).to(device)
print(f"ViT output shape: {vit_test(dummy).shape}")
del vit_test, dummy
```

```
ViT parameters: 1,222,410
ViT output shape: torch.Size([2, 10])
```

```
def get_resnet18(num_classes=10):
    """ResNet-18 modified for 32×32 CIFAR images."""
    model = resnet18(weights=None)
    # Replace first conv: 7×7 stride 2 -> 3×3 stride 1 for small images
    model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    model.maxpool = nn.Identity()  # Remove maxpool for 32×32
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

resnet_test = get_resnet18().to(device)
print(f"ResNet-18 parameters: {count_parameters(resnet_test):,}")
del resnet_test
```

```
ResNet-18 parameters: 11,173,962
```

```
class FocalLoss(nn.Module):
    """Focal Loss for handling class imbalance / hard examples."""
    def __init__(self, alpha=1.0, gamma=2.0, reduction='mean'):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss
        if self.reduction == 'mean':
            return focal_loss.mean()
        return focal_loss.sum()


def get_loss_fn(loss_name):
    """Return loss function by name."""
    if loss_name == 'cross_entropy':
        return nn.CrossEntropyLoss()
    elif loss_name == 'label_smoothing':
        return nn.CrossEntropyLoss(label_smoothing=0.1)
    elif loss_name == 'focal':
        return FocalLoss(alpha=1.0, gamma=2.0)
    else:
        raise ValueError(f"Unknown loss: {loss_name}")


def get_optimizer(model, opt_name, lr):
    """Return optimizer by name."""
    if opt_name == 'sgd':
        return optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                         weight_decay=1e-4)
    elif opt_name == 'rmsprop':
        return optim.RMSprop(model.parameters(), lr=lr, weight_decay=1e-4)
    elif opt_name == 'adam':
        return optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    else:
        raise ValueError(f"Unknown optimizer: {opt_name}")
```

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total


@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total


def train_model(model, train_loader, val_loader, criterion, optimizer,
                scheduler, num_epochs, run_name):
    """Full training loop with W&B logging."""
    best_val_acc = 0.0
    best_model_wts = copy.deepcopy(model.state_dict())
    history = {'train_loss': [], 'val_loss': [],
               'train_acc': [], 'val_acc': []}

    start_time = time.time()

    for epoch in range(num_epochs):
        train_loss, train_acc = train_one_epoch(
            model, train_loader, criterion, optimizer)
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        if scheduler is not None:
            scheduler.step()
```

```
        # Log to W&B
        wandb.log({
            "epoch": epoch + 1,
            "train_loss": train_loss,
            "train_acc": train_acc,
            "val_loss": val_loss,
            "val_acc": val_acc,
            "lr": optimizer.param_groups[0]['lr'],
        })

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['train_acc'].append(train_acc)
        history['val_acc'].append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_wts = copy.deepcopy(model.state_dict())

        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f"  Epoch {epoch+1:3d}/{num_epochs} | "
                  f"Train Loss: {train_loss:.4f} Acc: {train_acc:.2f}% | "
                  f"Val Loss: {val_loss:.4f} Acc: {val_acc:.2f}%")

    total_time = time.time() - start_time
    model.load_state_dict(best_model_wts)

    print(f"  Training time: {total_time:.1f}s | Best Val Acc: {best_val_acc:.2f}%")
    wandb.log({"best_val_acc": best_val_acc, "training_time_s": total_time})

    return model, history, total_time, best_val_acc
```

```
# This runs all 36 configurations (2 models × 2 aug × 3 loss × 3 opt).
# Estimated time on Colab GPU: ~3–5 hours.
# To do a quick test first, set QUICK_TEST = True below.

QUICK_TEST = False   # Set True to run only 1 config per model for sanity check

NUM_EPOCHS      = 20
BATCH_SIZE      = 128
LR_VIT          = 1e-3
LR_RESNET       = 0.01

MODEL_NAMES     = ['vit', 'resnet18']
AUG_OPTIONS     = [False, True]
LOSS_NAMES      = ['cross_entropy', 'label_smoothing', 'focal']
OPT_NAMES       = ['sgd', 'rmsprop', 'adam']

if QUICK_TEST:
    MODEL_NAMES = ['vit', 'resnet18']
    AUG_OPTIONS = [False]
    LOSS_NAMES  = ['cross_entropy']
    OPT_NAMES   = ['adam']
    NUM_EPOCHS  = 5

# Store all results
all_results = []

WANDB_PROJECT = "experiment10-vit-resnet"  # Change if you like

configs = list(iter_product(MODEL_NAMES, AUG_OPTIONS, LOSS_NAMES, OPT_NAMES))
print(f"Total configurations to run: {len(configs)}")

for i, (model_name, augment, loss_name, opt_name) in enumerate(configs):
    aug_tag = "aug" if augment else "noaug"
    run_name = f"{model_name}_{aug_tag}_{loss_name}_{opt_name}"
    print(f"\n{'='*60}")
    print(f"[{i+1}/{len(configs)}] {run_name}")
    print(f"{'='*60}")

    # Initialize W&B run
    wandb.init(
        project=WANDB_PROJECT,
        name=run_name,
        config={
            "model": model_name,
            "augmentation": augment,
            "loss_function": loss_name,
            "optimizer": opt_name,
            "epochs": NUM_EPOCHS,
            "batch_size": BATCH_SIZE,
            "lr": LR_VIT if model_name == 'vit' else LR_RESNET,
        },
        reinit=True,
    )

    # Data
    train_loader, val_loader, test_loader = get_dataloaders(
        augment=augment, batch_size=BATCH_SIZE)

    # Model
    if model_name == 'vit':
        model = VisionTransformer(
            img_size=32, patch_size=4, in_channels=3, num_classes=10,
            embed_dim=128, depth=6, num_heads=4, mlp_ratio=4.0, dropout=0.1
        ).to(device)
        lr = LR_VIT
```

```
        else:
            model = get_resnet18(num_classes=10).to(device)
            lr = LR_RESNET

        # Loss & optimizer
        criterion = get_loss_fn(loss_name)
        optimizer = get_optimizer(model, opt_name, lr)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS)

        # Train
        model, history, train_time, best_val = train_model(
            model, train_loader, val_loader, criterion, optimizer,
            scheduler, NUM_EPOCHS, run_name)

        # Test
        test_loss, test_acc = evaluate(model, test_loader, criterion)
        print(f"  >> Test Accuracy: {test_acc:.2f}%")

        wandb.log({"test_acc": test_acc, "test_loss": test_loss})

        # Save model checkpoint
        ckpt_path = f"checkpoints/{run_name}.pt"
        import os; os.makedirs("checkpoints", exist_ok=True)
        torch.save(model.state_dict(), ckpt_path)

        all_results.append({
            "run_name": run_name,
            "model": model_name,
            "augmentation": augment,
            "loss": loss_name,
            "optimizer": opt_name,
            "best_val_acc": best_val,
            "test_acc": test_acc,
            "train_time_s": train_time,
            "params": count_parameters(model),
        })

        wandb.finish()

    print("\n\nAll experiments completed!")
```